FOOD BUDDY — COMPREHENSIVE VIVA GUIDE

Campus Hostel Mess Management, Feedback & Institutional Governance System | Complete Technical Defense Manual

Project: Food Buddy (Hostel Mess Management)	**Author:** Omkar Manikire
Stack: Vanilla ES6+ HTML5/CSS3 + Supabase + Edge Functions + Vercel	**Target Audience:** External Examiner & Technical Evaluation Committee
Database: PostgreSQL 15 (Supabase) with 100% RLS	**Security Model:** 2FA OTP + Google OAuth + Pre-verified Whitelist

1. Executive Summary & Problem Motivation

The Problem: In collegiate and university campus hostels, mess operations suffer from critical systemic bottlenecks: (1) Paper-based complaint registers where hygiene, stale food, or contamination issues are easily ignored, manipulated, or destroyed; (2) High food wastage caused by lack of attendance forecasting; (3) Turnstile fraud and proxy meal claims where students share ID cards or take meals multiple times; (4) Subjective menu management where student dietary preferences and dish popularity are ignored; and (5) Complete lack of transparency for Hostel Wardens and Higher Authorities to enforce SLA accountability and penalties on private catering contractors.

The Solution (Food Buddy): Food Buddy is a modern, real-time, 3-tier hostel mess ecosystem connecting **Students**, **Mess Managers (Kitchen Supervisors)**, and **Higher Authorities (Hostel Wardens / Chief Wardens)**. The system enforces strict Row Level Security (RLS), multi-factor authenticated verification, dynamic anti-proxy QR attendance, SLA-enforced grievance tracking with automated 48-hour escalation to the Warden, daily FSSAI pre-meal inspection sign-offs, and quantitative caterer governance scorecards.

2. System Architecture & Technology Stack

Layer	Technologies Used	Engineering Purpose & Rationale
Frontend Presentation	Semantic HTML5, CSS3 Variables, Responsive CSS Grid/Flexbox, Vanilla ES6+ JavaScript, Tailwind CSS CDN	Lightweight, zero-build bundle requirement for browser execution. Native web standards ensure ultra-fast load times, zero framework bloat, and smooth mobile responsiveness.
Backend & BaaS	Supabase (Managed PostgreSQL 15+ engine)	Provides enterprise ACID relational storage, instant RESTful APIs (PostgREST), Realtime WebSocket engine, and declarative Row Level Security (RLS).
Database Security	PostgreSQL Row Level Security (RLS) + SECURITY DEFINER RPCs	Every table is locked at the database kernel level. Clients cannot read or mutate data across roles or outside their assigned tenancy, regardless of client-side code modification.

Serverless Logic	Supabase Edge Functions (Deno / TypeScript runtime)	Executes trusted privileged business logic: OTP generation/verification, SMTP email dispatch, Student ID whitelist validation, and OAuth token normalization without exposing master secrets.
Object Storage	Supabase Storage (`complaint-media` private bucket)	Stores encrypted grievance photo and video evidence (≤5MB images, ≤20MB videos). Access requires signed time-expiring URLs verified by RLS policies.
Hosting & DevOps	Vercel (Production Edge CDN) + GitHub Version Control	Automated CI/CD deployments from `main` branch. Custom `vercel.json` provides clean URL routing, security headers (CSP, XFO, nosniff), and cache invalidation.

3. Detailed Role Capabilities & Portals

A. Student Dashboard (`/student/`) — Resident Experience

- **Live Attendance & Dynamic QR Scan:** Scans mess turnstile QR code or inputs emergency alphanumeric token (`FB-LUNCH-2025`). Validated via `mark_attendance` RPC; duplicate scans in same meal slot are blocked.
- **Daily Meal Rating & Sentiment:** Rates today's breakfast, lunch, snacks, or dinner across Food Quality, Counter Hygiene, and Staff Courtesy (1 to 5 stars + optional comments). Unique constraint ensures 1 student = 1 rating per meal.
- **Grievance / Complaint Filing:** Submits ticket across 5 categories with mandatory 20+ character description and photo/video evidence. Instant tracking ID (`FB-YYYY-XXXXX`) generated. Rate-limited to max 5 complaints/hour.
- **Live Ticket Tracking:** Inspects real-time 4-step SLA timeline (`submitted` → `under_review` → `in_progress` → `resolved`), manager's mandatory resolution note, and Warden directives.
- **Menu & Nutritional Exploration:** Live display of today's menu, current active meal highlighted by time clock, estimated calories, and 7-day schedule rotation.
- **Food Improvement & Wishlist:** Submits recipe recommendations with oil/spice preference sliders and dish wishlist tags reviewed during Friday dining committee meetings.
- **Notices & Announcements:** Reads warden circulars with priority tags (Urgent, Feast, Normal) and live unread notification bell badge synced via `notice_reads` table.

B. Mess Manager Dashboard (`/manager/`) — Operations & Triage

- **Live Operational KPI Hub:** Real-time headcount turnout by meal (Breakfast, Lunch, Snacks, Dinner), active grievance count, and daily average sentiment score.
- **Dynamic QR Turnstile Kiosk (`/manager/qr.html`):** Fullscreen turnstile kiosk that displays cryptographic QR tokens refreshed automatically every 150 seconds to prevent photo sharing and proxy attendance.
- **Grievance Triage Queue (`/manager/complaints.html`):** Filters tickets by urgency/status, inspects student photographic evidence, transitions ticket status, and enforces a *mandatory resolution note* before marking resolved.
- **Pre-Meal Quality Checklist (`/manager/checklist.html`):** Mandatory FSSAI compliance verification signed off before every meal service (Taste approval, thermal well temperatures recorded, roti softness verified, Head Cook digital sign-off).
- **Weekly Menu Editor (`/manager/menu.html`):** 7-day weekly menu management with instant reactive updates pushed to student dashboards.
- **Attendance Ledger & CSV Export (`/manager/attendance.html`):** Searchable attendance ledger filtered by date, meal slot, and resident roll ID with 1-click CSV export for hostel billing.
- **Notice Publisher & Archive (`/manager/notices.html`):** Broadcasts operational circulars with auto-expiry dates and immutable audit archiving (hard deletes prohibited).

C. Higher Authority Dashboard (`/authority/`) — Supervisory Oversight

- **Executive KPI Overview:** Campus-wide grievance totals, overdue ticket counters (>48 hours), and overall student satisfaction metrics across all dining halls.
- **48-Hour Escalated Queue (`/authority/escalated.html`):** Automated queue tracking unresolved complaints. When a ticket exceeds 48h, it is automatically flagged escalated.
- **Demand Accountability Engine:** Warden issues binding administrative directives directly on escalated tickets (`demand_accountability` RPC). Broadcasts immediately to manager's triage view and student's tracking drawer.
- **Official Warden Directives (`/authority/directives.html`):** Publishes top-level institutional decrees featuring an official blue 'Authority' badge that cannot be overwritten or altered by mess managers.
- **Caterer Governance & Scorecard (`/authority/governance.html`):** Algorithmic compliance rating (0 to 100) based on student rating averages, resolved vs escalated ratio, and inspection standards. Logs statutory penalties and contractual deductions in `governance_actions`.
- **Immutable Campus Audit Trail (`/authority/audit.html`):** Read-only complete history of every complaint, timeline update, manager resolution note, and warden comment. SELECT-only access guarantees authority cannot tamper with historic data.

4. Database Schema & Row Level Security (RLS)

Database Philosophy: Security is enforced at the database level using PostgreSQL Row Level Security (RLS) and Postgres `SECURITY DEFINER` functions. Even if an attacker uses the browser DevTools to run arbitrary SQL or REST queries with the anon key, PostgreSQL rejects unauthorized operations with HTTP 403 / code 42501.

Table Name	Key Columns & Constraints	RLS Policy & Access Control
`profiles`	id (PK, FK auth.users), email, role ('student'|'mess_manager'|'authority'), student_id, otp_verified_until	Users can view own profile. Updates restricted; role and otp_verified_until protected by trigger from client tampering.
`valid_student_ids`	student_id (PK), is_used (bool), used_by (FK), batch_year	100% BLOCKED from client direct reads (0 rows returned). Read and mutated exclusively via backend Edge Functions during registration.
`attendance`	id, student_id, meal_date, meal_slot, nonce_used UNIQUE (student_id, meal_date, meal_slot)	Direct INSERT denied. Inserts permitted solely through `mark_attendance` RPC. Students view own check-ins; managers view all.
`qr_sessions`	id, active_token, meal_slot, expires_at, is_active	Client read completely blocked. Only manager can read via `get_current_qr()`; validated securely inside `mark_attendance`.
`complaints`	id, tracking_id, student_id, category, description, media_paths, status, escalated	Students can SELECT own complaints. Managers can SELECT all and UPDATE status (with note). Authorities have read-only audit access.
`complaint_history`	id, complaint_id, status, note, actor_role, changed_at	Append-only audit trail. Triggered automatically on ticket creation, manager triage, and authority accountability directives.
`ratings`	id, student_id, meal_date, meal_slot, food_rating, hygiene_rating, staff_rating UNIQUE (student_id, meal_date, meal_slot)	Students view own ratings. Managers access anonymized 30-day statistical aggregates via `get_manager_ratings_analytics` RPC.
`quality_checklists`	id, inspection_date, meal_slot, tasted_and_approved, head_cook_name, remarks	Managers sign off via `signoff_quality_checklist` RPC. Students and authorities have read access to verify safety before meals.

`accountability_comments`	id, complaint_id, authority_id, directive_text, created_at	Only users with role 'authority' can insert via `demand_accountability` RPC. Visible to students and managers.
`governance_actions`	id, caterer_name, action_type, amount_deducted, reason_note, authority_id	Higher Authority exclusive table. Records statutory vendor penalties and contract renewal decrees. Blocked from student access.

5. Advanced Engineering Workflows

A. 2-Factor Authentication (OTP) & Verification Life Cycle

1. **Password Verification:** User enters email and password into Supabase Auth. At this stage, RLS policies deny access to application data because `profiles.otp_verified_until` is NULL or expired.
2. **OTP Edge Function (`send-login-otp`):** The client calls `send-login-otp`. The function generates a cryptographically random 6-digit integer, hashes it, stores it in server table `login_otps` with a 5-minute TTL, and sends it via SMTP. A 30-second cooldown is enforced to prevent spam.
3. **OTP Verification (`verify-login-otp`):** User submits code. Edge function checks hash and attempt counter. If incorrect, attempt counter decrements; at 3 wrong attempts, the code is locked out. On success, `profiles.otp_verified_until` is set to `now() + interval '12 hours'`.
4. **RLS Grant:** Database function `public.is_verified_user()` checks `auth.uid() = id AND otp_verified_until > now()`. The student is now authorized to query data for 12 hours.
5. **Instant Revocation:** Upon logout, `clear_otp_verification()` immediately resets `otp_verified_until` to NULL, blocking any reused tokens.

B. Dynamic Anti-Proxy QR Turnstile Attendance

• **The Fraud Vulnerability:** In traditional QR systems, students take a screenshot of the QR code and message it to roommates to mark proxy attendance from their dormitory beds.
• **Food Buddy Mitigation:** The Mess Manager QR screen runs an auto-refreshing kiosk that polls `get_current_qr()`. The server generates a unique cryptographic token (`FB-QR-LUNCH-XXXXXX`) valid for only **150 seconds**. When a student scans, `mark_attendance(p_token)` verifies that the token is currently active and unexpired. Once marked, the database enforces `UNIQUE(student_id, meal_date, meal_slot)`, making duplicate entry impossible.

C. Grievance Lifecycle & Automated 48-Hour Escalation

• **Submission:** Student files grievance with category and photos. `create_complaint` generates tracking ID `FB-2026-XXXXX` and inserts audit log.
• **Manager Triage:** Mess Manager reviews ticket. Changing status to `resolved` requires a mandatory descriptive `p_resolution_note`.
• **Automated Escalation:** An automated cron / supervisory routine executes `process_complaint_escalations()`. Any complaint with `status IN ('submitted', 'under_review', 'in_progress')` and `created_at < now() - interval '48 hours'` is automatically flagged with `escalated = true` and `escalated_at = now()`.
• **Warden Intervention:** Escalated complaints immediately appear in the Warden's priority queue. The Warden executes `demand_accountability()`, which writes a binding decree to `accountability_comments` and notifies the manager.

D. Google OAuth & Institutional Onboarding Flow

• When a student signs in with Google, `complete-google-login` verifies the OAuth token. If the user's email is not yet mapped in `profiles`, the function returns `needs_onboarding: true`.
• The student is redirected to `/onboarding.html`, where they must input their official Institutional Student ID (e.g. `STU-2024-015`).
• `complete-google-onboarding` checks the ID against `valid_student_ids`. If valid and unused, the ID is flagged used, the student profile is created, and access is unlocked. If an outsider attempts to use a fake ID or an already-claimed ID, the transaction is rejected.

6. Top 35 External Examiner Viva Questions & Answers

Q1: What is Food Buddy, and what motivated you to develop this project?

Answer: Food Buddy is a full-stack, 3-tier campus hostel mess management and institutional accountability system. It was designed to replace paper registers and unmonitored catering practices by digitizing meal feedback, preventing attendance fraud via dynamic QR tokens, and enforcing contractual catering accountability through automated 48-hour grievance escalations to the Hostel Warden.

Q2: Why did you choose a 3-tier user hierarchy (Student, Manager, Authority)?

Answer: A 2-tier system (Student and Manager) creates an accountability deadlock: if the manager ignores bad food or unhygienic conditions, the student has no recourse. By introducing the Higher Authority (Hostel Warden / Dean), we establish an independent oversight layer that audits resolution metrics, reviews unresolved tickets, and enforces penalties on the catering contractor.

Q3: Why did you build the frontend in Vanilla ES6+ rather than React, Angular, or Next.js?

Answer: Using standard modern Web Components, semantic HTML5, and vanilla ES6 modules eliminated node build-chain complexity, reduced runtime bundle sizes to zero kilobytes, guaranteed lightning-fast First Contentful Paint (<200ms), and ensured that every page operates natively on any mobile browser without heavy client-side JavaScript hydration overhead.

Q4: Why choose Supabase and PostgreSQL over Firebase or MongoDB?

Answer: Mess management is inherently relational: students attend specific meal slots, meals belong to menus, complaints generate chronological audit histories, and ratings link to specific dates. PostgreSQL provides strict ACID transactions, foreign key cascades, check constraints, and above all, Row Level Security (RLS) built into the database kernel, which NoSQL databases cannot provide natively.

Q5: What is the architectural role of Supabase Edge Functions?

Answer: Edge Functions run on Deno at edge locations worldwide. They serve as trusted serverless microservices for privileged operations that must never happen in client code: verifying student ID whitelists, hashing and validating 6-digit OTPs, enforcing rate limits, sending SMTP emails, and managing Google OAuth onboarding.

Q6: How does Row Level Security (RLS) work in your application?

Answer: RLS is a PostgreSQL feature that evaluates security policies on every single SQL query executed by the database. In Food Buddy, RLS policies check `auth.uid() = student_id` and evaluate `public.is_verified_user()`. If a student attempts to query `SELECT * FROM complaints;`, Postgres filters the query at the engine level and returns only records belonging to that student.

Q7: Can a student alter their role from 'student' to 'mess_manager' using browser DevTools?

Answer: No. Role escalation is blocked on three distinct layers: (1) Client REST calls cannot write to the `role` column due to column-level RLS restrictions; (2) A database trigger `prevent_profile_role_escalation` checks if `OLD.role <> NEW.role` and throws an exception; (3) All privileged RPC functions inspect `public.get_auth_role() = 'mess_manager'` on the server.

Q8: What is the difference between SUPABASE_ANON_KEY and SUPABASE_SERVICE_ROLE_KEY?

Answer: The `SUPABASE_ANON_KEY` is a public publishable key embedded in client browser JavaScript (`public/js/config.js`). It is restricted by Row Level Security policies. The `SUPABASE_SERVICE_ROLE_KEY` is a secret master key that bypasses all RLS policies. It is used exclusively in backend Edge Functions and local seeding scripts, and is strictly gitignored so it is never exposed in git or client code.

Q9: What is the purpose of the is_verified_user() function?

Answer: `is_verified_user()` is a database security helper function. It checks if the requesting user's profile has `otp_verified_until > now()`. This enforces 2-Factor Authentication: even if someone steals a user's password, querying database tables returns 0 rows until they complete the 6-digit email OTP verification.

Q10: How do you prevent brute-force attacks on your OTP verification system?

Answer: Our `verify-login-otp` Edge Function tracks an integer attempt counter in the `login_otps` table. If a user enters an incorrect OTP 3 times, the OTP is permanently invalidated and deleted. Furthermore, `send-login-otp` enforces a 30-second cooldown timer between code dispatches to stop spamming.

Q11: How does your application prevent outsiders from registering with the hostel portal?

Answer: We maintain a pre-approved database table called `valid_student_ids` populated by the hostel administration. When a student registers, a database trigger or Edge Function verifies that the entered student ID exists and has `is_used = false`. Once registered, it is marked `is_used = true`. Unrecognized or duplicate IDs are immediately rejected with HTTP 400.

Q12: How does the rotating QR turnstile code prevent proxy attendance?

Answer: Instead of a static printed QR code that students can photograph and share over WhatsApp, the manager displays a dynamic QR kiosk. The token (`FB-QR-LUNCH-XXXXXX`) rotates every 150 seconds. When a student scans, `mark_attendance` checks if `expires_at > now()`. A photo taken earlier will fail validation within 2.5 minutes.

Q13: What happens if a student tries to scan attendance twice for the same lunch?

Answer: The `attendance` table has a composite unique constraint: `UNIQUE (student_id, meal_date, meal_slot)`. If a student scans a second time on the same date and slot, PostgreSQL throws a unique constraint violation, and the RPC returns a user-friendly error: 'Attendance already marked for Lunch on [Date].'

Q14: What if a student's phone camera is broken or the QR display fails?

Answer: The system provides an emergency manual fallback token field. The student enters the active token displayed on the turnstile monitor (e.g. `FB-LUNCH-2025`). The backend validates the token using the same single-use meal logic.

Q15: Describe the lifecycle of a complaint in Food Buddy.

Answer: A complaint begins in state `submitted`. When the kitchen supervisor views and begins investigating, it transitions to `under_review`. When corrective action is initiated (e.g. replacing a batch or recalibrating cooking temperature), it becomes `in_progress`. Upon resolution, the manager must input a mandatory resolution note to mark it `resolved`. Every transition appends an immutable entry to `complaint_history`.

Q16: How does the 48-hour automated grievance escalation work?

Answer: A database procedure `process_complaint_escalations()` queries all complaints where `status IN ('submitted', 'under_review', 'in_progress')` and `created_at < now() - interval '48 hours'`. It automatically updates `escalated = true` and logs an escalation event in `complaint_history`. This query is scheduled periodically via pg_cron or triggered during authority oversight routines.

Q17: What is the 'Demand Accountability' action?

Answer: When an issue escalates or involves a health hazard (e.g. foreign particle in food), the Hostel Warden can invoke the `demand_accountability` RPC. This records a formal decree in `accountability_comments` and appends a notice to the manager's review queue. Mess managers cannot delete or dismiss Warden directives.

Q18: What is the Caterer Governance Scorecard and how is it calculated?

Answer: The Caterer Governance Scorecard is a quantitative metric (0 to 100) computed by `get_caterer_scorecard()`. It awards points based on: (1) Average student food satisfaction ratings (up to 40 pts), (2) Grievance resolution efficiency (ratio of resolved vs unresolved, up to 30 pts), (3) Zero overdue escalations (up to 20 pts), and (4) Pre-meal checklist compliance (up to 10 pts). Deductions are logged when statutory penalties are issued.

Q19: What is the Pre-Meal Quality Checklist, and why is it mandatory?

Answer: In accordance with FSSAI institutional dining guidelines, food must be inspected before service begins. In `/manager/checklist.html`, the Head Cook must confirm: (1) Dishes tasted and approved, (2) Food holding temperature recorded above 65°C, (3) Kitchen sanitation verified, and (4) Phulkas/Rotis inspected for softness. The Head Cook's digital sign-off is logged in `quality_checklists`.

Q20: How are media files (photos/videos of food complaints) stored?

Answer: Files are uploaded to a private Supabase Storage bucket named `complaint-media`. Photos are capped at 5MB and videos at 20MB. Because the bucket is private, files cannot be accessed via public URLs; the client requests short-lived signed URLs generated through authenticated Supabase storage policies.

Q21: How does the meal rating system prevent duplicate submissions or ballot stuffing?

Answer: The `ratings` table enforces a database constraint: `UNIQUE (student_id, meal_date, meal_slot)`. A student can submit exactly one review per meal slot per day. Attempting to submit multiple reviews will trigger an HTTP 409 duplicate rejection from Postgres.

Q22: How does the student food improvement / suggestion system work?

Answer: Students can submit recipe ideas, dish wishlist items (e.g. Paneer Butter Masala, Brown Rice), and indicate their desired spice and oil levels using interactive sliders. These suggestions are compiled into weekly reports for the Friday Dining Committee meeting.

Q23: How does real-time communication work in Food Buddy?

Answer: Food Buddy integrates Supabase Realtime via WebSockets (`wss://*.supabase.co`). When a manager resolves a grievance, Supabase broadcasts a PostgreSQL Change Data Capture (CDC) event over WebSockets. The student's open tracking drawer updates its status pill and displays the manager's note without page reloading.

Q24: What happens if a user's browser loses internet connectivity?

Answer: All fetch operations in `/js/supabase.js` and `/js/auth.js` are wrapped in try-catch handlers with toast notifications. If an API request fails, a visible red alert banner notifies the user of the network disconnection rather than crashing silently.

Q25: What automated test suites exist in the codebase?

Answer: The project features 6 distinct test suites executed via `npm test`: (1) `npm run check` (static syntax & structural integrity across all 88 files), (2) `test:rls` (40 security tests covering OTP and outsider rejection), (3) `test:student` (22 tests for student workflows), (4) `test:manager` (27 tests for kitchen triage and QR rotation), (5) `test:authority` (33 tests for directives and scorecards), and (6) `test:security` (privilege escalation and secret leak checks). All 160+ tests pass with 0 failures.

Q26: How did you test cross-role isolation?

Answer: In `test-phase4-security.mjs`, we simulate a student session attempting to invoke `get_manager_overview_kpis()`, `get_current_qr()`, and `demand_accountability()`. The test verifies that the database throws error code `P0001` or `42501` (Access Denied) in 100% of unauthorized attempts.

Q27: If 5,000 students scan the attendance QR within a 15-minute window, how will the database handle the load?

Answer: The `mark_attendance` RPC is an indexed single-row insert evaluated against a primary key and indexed unique constraint. PostgreSQL can easily execute 10,000+ indexed inserts per second on modest hardware. Because Supabase uses connection pooling (PgBouncer) on port 6543, transient connection spikes are queued efficiently without exhausting PostgreSQL backend processes.

Q28: How do you prevent SQL Injection attacks?

Answer: Food Buddy uses Supabase PostgREST and parameterized stored procedures (`$1, $2, ...`). User inputs are never concatenated directly into raw SQL strings. PostgreSQL treats parameters strictly as typed data, completely neutralizing SQL injection.

Q29: What security headers are deployed on Vercel?

Answer: In `vercel.json`, we enforce: (1) `Content-Security-Policy` restricting script execution to approved CDNs, (2) `X-Frame-Options: SAMEORIGIN` preventing clickjacking in iframes, (3) `X-Content-Type-Options: nosniff` preventing MIME-type sniffing, and (4) `Strict-Transport-Security` forcing HTTPS.

Q30: Why is the .env file excluded from GitHub?

Answer: The `.env` file contains server-side secrets like `SUPABASE_SERVICE_ROLE_KEY` and SMTP credentials. If committed to GitHub, automated scanners and bad actors could extract the keys and gain complete root access to the database. The client frontend only requires the public `SUPABASE_ANON_KEY`, which is safely bundled via `public/js/config.js`.

Q31: What happens if an expired OTP is submitted?

Answer: `verify-login-otp` checks `expires_at > now()`. If the 5-minute time-to-live has lapsed, the function rejects the code with HTTP 400: 'Verification code has expired. Please request a new one.'

Q32: Can a student delete an existing notice or circular?

Answer: No. RLS policy on table `notices` restricts DELETE operations. In fact, even managers cannot hard-delete notices: notices can only be edited or archived (`is_archived = true`) to preserve historical audit logs.

Q33: How does the system handle students who are absent from hostel for holidays?

Answer: Students simply do not generate attendance records for those meal dates. The Mess Manager attendance table aggregates turnout numbers, allowing the kitchen to compare expected versus actual headcount and adjust raw ingredient procurement accordingly.

Q34: How does Food Buddy handle mobile responsiveness across different device sizes?

Answer: The UI is built using responsive CSS Grid with `repeat(auto-fit, minmax(min(100%, 260px), 1fr))` and flexible containers with explicit `min-width: 0;` to prevent grid blowout. It includes a responsive sidebar for desktop (1280px), collapsible drawers for tablets (768px), and bottom tab bars for mobile devices (375px).

Q35: What future enhancements would you propose for Food Buddy 2.0?

Answer: Future iterations could incorporate: (1) Predictive AI for food quantity forecasting based on historical turnout patterns, (2) Biometric facial recognition integration for turnstile gates, (3) Automatic caterer billing calculation linked directly to real attendance headcounts, and (4) Push notifications via Progressive Web App (PWA) service workers.

7. Quick Revision Flashcards & Glossary

Term	Definition for Viva
Row Level Security (RLS)	A PostgreSQL security subsystem that executes per-row access control policies based on database roles and session context (`auth.uid()`).
SECURITY DEFINER	A PostgreSQL function property that executes with the privileges of the function's creator rather than the caller, allowing safe, controlled execution of privileged operations.
PostgREST	The web server engine within Supabase that inspects PostgreSQL schema and automatically generates clean RESTful APIs with Swagger documentation.
Change Data Capture (CDC)	The mechanism by which Supabase Realtime listens to PostgreSQL's write-ahead log (WAL) and broadcasts insert/update events to client WebSockets.
Nonce / Cryptographic Token	A single-use or time-bounded random string (`FB-QR-LUNCH-XXXXXX`) that prevents replay attacks in the dynamic QR attendance turnstile.
SLA (Service Level Agreement)	Contractual timeline thresholds (e.g. 2h inspection for hygiene hazard, 24h for standard complaints, 48h automatic warden escalation).

Examiner Tip: *When presenting your project, start with the live demo: show the Student submitting a complaint with an image, switch to the Manager dashboard to show it arriving in real-time, demonstrate the mandatory resolution note, and show the Higher Authority dashboard enforcing accountability. Emphasize that security is in the database kernel (RLS), not just the UI!*